

گزارش Main+Baseline

پروژه تشخیص تقلب در تراکنش‌های مالی (Fraud Detection)

بخش اول: تغییرات بخش داده (Data) و دلایل آن‌ها

با مقایسه دقیق نوت‌بوک اولیه (Main) با نسخه به‌روزشده (Main+Baseline)، تغییرات زیر در بخش داده شناسایی شد:

۱. حذف رکوردهای تکراری (Duplicates) پیش از تقسیم داده

در بخش «بازرسی کیفیت داده» (Data Quality Inspection) در نسخه اولیه، حدود ۱۰۸۱ ردیف تکراری شناسایی شده بود اما هرگز حذف نمی‌شد؛ فقط گزارش می‌شد. در نسخه جدید، این خط اضافه شده است:

```
df2 = df2.drop_duplicates().reset_index(drop=True)
```

چرا: اگر یک ردیف تکراری وجود داشته باشد و یک نسخه از آن در train و نسخه دیگر در test قرار بگیرد، مدل عملاً یکی از نمونه‌های تست را «حفظ کرده» می‌بیند. این پدیده نشت داده (Data Leakage) نام دارد و باعث می‌شود متریک‌های ارزیابی روی داده تست، خوش‌بینانه‌تر از واقعیت به نظر برسند. حذف تکراری‌ها پیش از train_test_split این ریسک را از بین می‌برد.

۲. افزودن stratify=y به تقسیم train/test

• قبل: `train_test_split(x, y, test_size=۰.۲, random_state=۴۲)`

• بعد: `train_test_split(x, y, test_size=۰.۲, random_state=۴۲, stratify=y)`

چرا: با توجه به عدم توازن شدید کلاس‌ها (تقلب فقط حدود ۰.۱۷٪ از داده است)، بدون stratify ممکن است به‌طور تصادفی نسبت تقلب در train و test تفاوت محسوسی داشته باشد. `stratify=y` تضمین می‌کند که نسبت کلاس‌ها در هر دو مجموعه حفظ شود و ارزیابی روی مجموعه تست، معتبرتر و قابل‌مقایسه‌تر باشد.

۳. تغییر شکل متغیر هدف (y)

• قبل: `y = df2["Class"].values.reshape(-۱, ۱)` → آرایه NumPy دوبعدی

• بعد: `y = df2["Class"]` → یک Series پانداس (یک‌بعدی)

چرا: ساده‌سازی کد در ادامه مسیر (نیازی به `ravel` مکرر نیست)؛ هرچند در برخی سلول‌های باقی‌مانده هنوز `ravel` به‌صورت احتیاطی فراخوانی می‌شود که در خروجی‌ها هم به‌صورت FutureWarning دیده می‌شود.

۴. ذخیره‌سازی رسمی تقسیم train/test برای کل تیم

```
joblib.dump({
    "x_train": x_train, "x_test": x_test,
    "y_train": y_train, "y_test": y_test,
    "feature_columns": x.columns.tolist(),
    }, "train_test_split_final.joblib")
```

چرا: این پروژه بین چند نفر/چند نوت‌بوک (Baseline, XGBoost/LightGBM, Feature Importance) تقسیم شده است. ذخیره تقسیم نهایی داده تضمین می‌کند که همه مدل‌ها روی دقیقاً همان ردیف‌های train/test آموزش و ارزیابی می‌شوند تا مقایسه بین مدل‌ها منصفانه و قابل‌اعتماد باشد.

۵. افزودن لایه Cache برای مراحل سنگین پیش‌پردازش

دو مرحله زمان‌بر cache شدند:

- نمونه‌گیری NCR (NeighbourhoodCleaningRule) روی x_train → ذخیره در cache_x_train prossed.joblib

- رتبه‌بندی Mutual Information → ذخیره در cache_mi_ranking.joblib

هر دو Cache با کنترل source_shape (شکل داده ورودی) معتبرسنجی می‌شوند؛ اگر x_train تغییر کند (مثلاً به دلیل حذف تکراری‌ها یا افزودن stratify)، Cache به‌طور خودکار نامعتبر شده و دوباره محاسبه می‌شود. چرا: اجرای NeighbourhoodCleaningRule و mutual_info_classif روی این حجم داده بسیار کند است. Cache کردن نتایج، زمان اجرای مجدد نوت‌بوک را به‌شدت کاهش می‌دهد، بدون این‌که ریسک استفاده از داده قدیمی/نامعتبر را به همراه داشته باشد.

۶. بازنویسی حلقه ارزیابی «تعداد ویژگی» بر پایه Mutual Information

قبل: در هر تکرار حلقه ($k=1..19$) دوباره `SelectKBest(score_func=mual_info_classif, k=k).fit(...)` اجرا می‌شد؛ یعنی MI تا ۱۹ بار محاسبه می‌شد.

بعد: رتبه‌بندی MI فقط یک‌بار (از طریق Cache) محاسبه می‌شود؛ سپس در هر k ، به‌سادگی k ویژگی برتر از لیست رتبه‌بندی‌شده انتخاب می‌شود (`ranked_features[:k]`). همچنین تعداد fold های Cross-Validation در این بخش از ۳ به ۵ افزایش یافته تا تخمین‌ها پایدارتر باشند.

چرا: افزایش سرعت اجرا؛ منطق نهایی مشابه است اما بدون محاسبات تکراری.

۷. مستندسازی صریح یک محدودیت شناخته‌شده (Caveat) درباره نشت احتمالی داده

در نسخه جدید یک یادداشت رسمی اضافه شده که می‌گوید چهار روش انتخاب ویژگی (Correlation, Mutual Information, ANOVA, Tree-Based) در ابتدای نوت‌بوک روی x/y کامل (یعنی پیش از تقسیم train/test) محاسبه شده‌اند، نه فقط روی x_{train} . این یعنی داده‌های x_{test} هم در انتخاب ویژگی‌های مهم تأثیر داشته‌اند — یک نوع نشت داده خفیف.

چرا مستند شده نه رفع شده: تیم تصمیم گرفته فعلاً با همین خط لوله ادامه دهد و فقط در صورت مشاهده نشانه آشکار خطا (مثلاً افت شدید عملکرد روی داده واقعی)، این بخش را با یک اجرای «بدون نشت» بازبینی کند. این شفافیت برای تیم بعدی (مسئول XGBoost/LightGBM) در بخش Handoff نیز تکرار شده است.

۸. فعال‌سازی ذخیره نمودارهای EDA

خطوطی مانند plt.savefig(...) که در Main به‌صورت کامنت بودند، در نسخه جدید فعال شده‌اند و نمودارهای EDA (توزیع کلاس، توزیع ویژگی‌ها، Skewness) واقعاً روی دیسک در مسیر `reports/figures/..` ذخیره می‌شوند — احتمالاً برای استفاده در گزارش نهایی.

جمع‌بندی بخش اول

تغییر	هدف اصلی
حذف Duplicates قبل از Split	جلوگیری از نشت داده بین train/test
افزودن stratify=y	حفظ نسبت کلاس‌ها در train/test با وجود عدم توازن شدید
ذخیره رسمی Split	یکسان‌سازی داده بین اعضای تیم/مدل‌های مختلف
Cache کردن NCR و MI	افزایش سرعت اجرای مجدد نوت‌بوک
بازنویسی حلقه ارزیابی ویژگی	حذف محاسبات تکراری MI
مستندسازی نشت احتمالی در Feature Importance	شفافیت و آگاهی تیم از یک محدودیت شناخته‌شده

۱. معماری Pipeline

مدل پایه بر اساس پایپ‌لاین قبلاً ساخته‌شده در نوت‌بوک بنا شده است:

QuantileTransformer (مقیاس‌بندی روی همه ۳۰ ویژگی) ↓
NeighbourhoodCleaningRule (نمونه‌گیری برای رفع عدم توازن) ↓
Logistic Regression (max_iter=2000)

نکات طراحی مهم که در گزارش به‌صراحت مستند شده‌اند:

- `class_weight` عمداً تنظیم نشده، چون `NeighbourhoodCleaningRule` از قبل کلاس‌ها را متوازن می‌کند؛ تنظیم هم‌زمان هر دو باعث «تصحیح دوباره» (`double-correction`) نامتوازن‌سازی می‌شد.
- معیار اصلی انتخاب مدل، `PR-AUC` (`average_precision`) است، نه `roc_auc` یا `accuracy`؛ چون با وجود عدم توازن شدید کلاس‌ها، `ROC-AUC` می‌تواند به‌طور گمراه‌کننده‌ای بالا باشد حتی برای مدل‌های ضعیف، درحالی‌که `PR-AUC` حساسیت بیشتری به عملکرد روی کلاس اقلیت (تقلب) دارد.
- انتخاب آستانه تصمیم‌گیری (`Threshold`) به‌جای مقدار پیش‌فرض ۰.۵، از طریق منحنی `Precision-Recall` و بیشینه‌سازی `F1` روی پیش‌بینی‌های `Out-of-Fold` انجام شده است — رویکردی دقیق‌تر از استفاده کورکورانه از آستانه ۰.۵.

۲. نتایج اعتبارسنجی متقابل (۵-Fold Stratified CV روی train)

متریک	Train	Valid (میانگین)	انحراف معیار Valid
Accuracy	۰.۹۹۹۴	۰.۹۹۹۴	۰.۰۰۰۱ ±
Precision	۰.۸۵۵۰	۰.۸۴۷۴	۰.۰۳۸۵ ±
Recall	۰.۷۹۸۹	۰.۷۹۰۹	۰.۰۷۱۵ ±
F1	۰.۸۲۵۹	۰.۸۱۵۷	۰.۰۳۸۶ ±
ROC-AUC	۰.۹۸۶۷	۰.۹۸۴۴	۰.۰۱۰۱ ±
Average Precision (PR-AUC)	۰.۷۸۷۱	۰.۷۸۵۱	۰.۰۴۲۲ ±

تحلیل: فاصله کم بین مقادیر Train و Valid در همه متریک‌ها نشان می‌دهد مدل به شدت Overfit نشده است. اما انحراف معیار نسبتاً بالای Recall (± 0.0715) و Average Precision (± 0.0422) بین fold ها طبیعی است، چون تعداد نمونه‌های تقلب در هر fold بسیار کم است (چند ده نمونه) و این باعث نوسان بالای متریک‌هایی می‌شود که مستقیماً به کلاس اقلیت وابسته‌اند.

۳. انتخاب آستانه (Threshold) بهینه

```
Best threshold (max F1, out-of-fold): 0.4918
Precision @ threshold: 0.8436
Recall @ threshold: 0.7989
F1 @ threshold: 0.8207
```

آستانه بهینه (۰.۴۹۱۸) به مقدار پیش فرض ۰.۵ بسیار نزدیک است، که نشان می‌دهد مدل به‌طور طبیعی خروجی احتمالی نسبتاً کالیبره‌شده‌ای تولید می‌کند و نیازی به جابه‌جایی زیاد آستانه نبوده است.

۴. ارزیابی نهایی روی مجموعه تست (Held-Out)

```
PR-AUC (test): 0.7036
```

	precision	recall	f1-score	support
0	0.9995	0.9998	0.9997	56651
1	0.8816	0.7053	0.7836	95
accuracy			0.9993	56746
macro avg	0.9405	0.8526	0.8916	56746
weighted avg	0.9993	0.9993	0.9993	56746

```
Confusion Matrix:
```

```
[[56642  9]
 [ 28  67]]
```

تحلیل تفصیلی

- از ۹۵ تراکنش تقلبی واقعی در مجموعه تست، مدل ۶۷ مورد را درست شناسایی کرده (Recall $\approx$ ۷۰.۵٪) و ۲۸ مورد را از دست داده (False Negative).
 - در مقابل، از میان ۵۶'۶۵۱ تراکنش سالم، فقط ۹ مورد به اشتباه به عنوان تقلب علامت گذاری شده اند (False Positive) — یعنی نرخ هشدار کاذب بسیار پایین (نزدیک به صفر).
 - Precision کلاس تقلب ۸۸.۱۶٪ است؛ یعنی از هر تراکنشی که مدل «تقلب» اعلام می کند، حدود ۸۸٪ واقعاً تقلب هستند — نکته مهمی برای کاربرد عملی، چون هشدارهای کاذب کمتر یعنی هزینه عملیاتی و آزار مشتری کمتر.
- افت Recall نسبت به CV: در Cross-Validation، میانگین Recall حدود ۷۹.۰۹٪ برآورد شده بود، اما در تست نهایی به ۷۰.۵۳٪ رسیده است. همچنین PR-AUC از ۰.۷۸۵۱ (CV) به ۰.۷۰۳۶ (تست) کاهش یافته. این افت به دو دلیل محتمل قابل توجیه است:
- (۱) تعداد نمونه های تقلب در تست فقط ۹۵ مورد است؛ با این حجم کم، هر چند نمونه اشتباه، درصد های Recall/Precision را به شدت نوسان می دهد (واریانس بالا، نه لزوماً افت واقعی عملکرد).
 - (۲) آستانه بهینه از روی داده Out-of-Fold train انتخاب شده و ممکن است دقیقاً روی توزیع داده تست بهینه نباشد؛ این یک خوش بینی خفیف در تخمین CV نسبت به عملکرد واقعی روی داده کاملاً دیده نشده است.
- با این حال، به طور کلی مدل Baseline عملکرد قابل قبولی دارد: Accuracy نزدیک ۱۰۰٪ (که به دلیل عدم توازن شدید داده معیار خوبی نیست)، اما مهم تر، F_1 کلاس تقلب برابر ۰.۷۸۳۶ و Macro F_1 برابر ۰.۸۹۱۶ نشان دهنده تعادل معقول بین شناسایی تقلب و کنترل هشدار کاذب است.

۵. صادرات مدل و مستندسازی برای تیم بعدی

- مدل نهایی همراه با نام مدل و آستانه انتخابی در فایل baseline_logreg.joblib ذخیره شده است. یک بخش «تحويل به تیم بعدی» (Handoff) به نوت بوک اضافه شده که مشخص می کند:
- از همان train_test_split_final.joblib همان scaler_transformer و همان NeighbourhoodCleaningRule (به عنوان بهترین روش از میان ۵ روش نمونه گیری آزموده شده) باید استفاده شود.
- معیار بهینه سازی برای تیوینگ مدل های بعدی (XGBoost/LightGBM با Optuna) باید average_precision باشد، نه accuracy.
- فعلاً باید از هر ۳۰ ویژگی استفاده شود؛ کاهش به ۱۹/۱۶ ویژگی هنوز «موقت» و منوط به بازبینی بدون نشت داده است.

● جمع‌بندی بخش دوم

مدل Baseline (رگرسیون لجستیک روی پایپ‌لاین QuantileTransformer + NCR) با انتخاب آستانه مبتنی بر منحنی PR، به Precision بالا (۸۸٪) و Recall معقول (۷۰٪) روی داده تست کاملاً دیده‌نشده دست یافته است. این عملکرد به‌عنوان یک نقطه مرجع (benchmark) مناسب برای مقایسه با مدل‌های پیچیده‌تر (XGBoost، LightGBM) در مراحل بعدی پروژه در نظر گرفته شده و مستندات لازم برای تکرارپذیری کامل کار (داده، پایپ‌لاین، معیار ارزیابی) به‌صورت شفاف در اختیار تیم بعدی قرار گرفته است.